Zellic

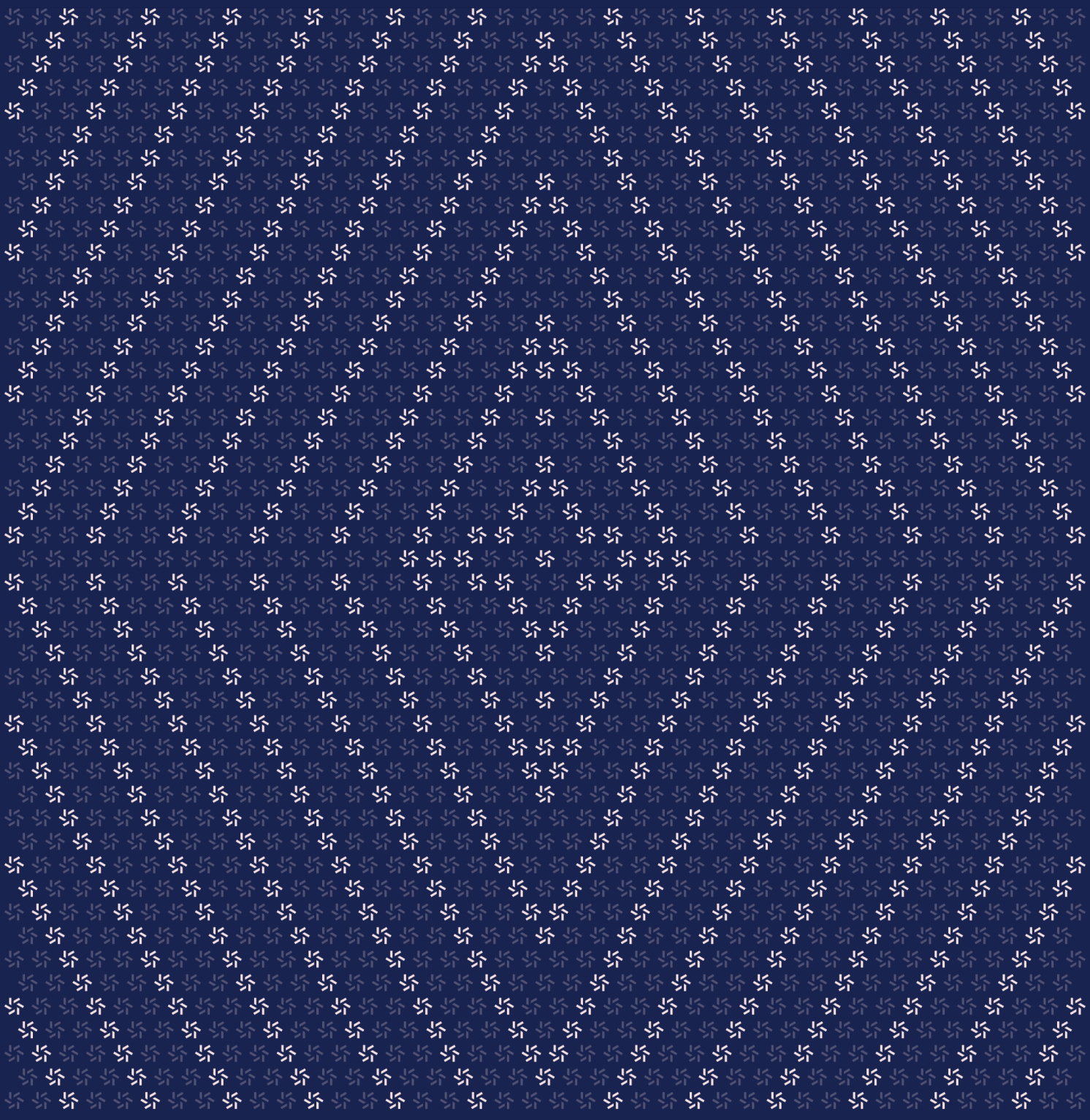
**Prepared for**
OKX

**Prepared by**
**Qingying Jie**
**Varun Verma**
Zellic

**December 23, 2025**

# xAsset

## Smart Contract Security Assessment



# Contents

## About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

# 1. Overview

## 1.1. Executive Summary

Zellic conducted a security assessment for OKX from December 8th to December 9th, 2025. During this engagement, Zellic reviewed xAsset's code for security vulnerabilities, design issues, and general weaknesses in security posture.

## 1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Do key functions have appropriate access control?
- Could users in the denylist send or receive tokens?
- Are there any operations that could potentially lead to a denial-of-service attack?

## 1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Off-chain programs
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

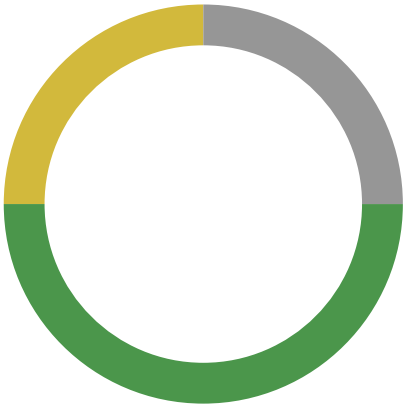
## 1.4. Results

During our assessment on the scoped xAsset contracts, we discovered four findings. No critical issues were found. One finding was of medium impact, two were of low impact, and the remaining finding was informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of OKX in the Discussion section (4. ↗).

## Breakdown of Finding Impacts

| Impact Level | Count |
|---|---|
| Critical | 0 |
| High | 0 |
| Medium | 1 |
| Low | 2 |
| Informational | 1 |



# 2. Introduction

## 2.1. About xAsset

OKX contributed the following description of xAsset:

> xAssets are centralized issued ERC-20 tokens backed by OKX, containing 2 categories:
>
> 1. Wrapped Tokens (1:1 Backed). Direct on-chain representations of underlying assets with guaranteed 1:1 backing by OKX reserves. Example: xBTC, xETH, xSOL.
> 2. Liquid Staking Tokens (Yield-Bearing). Tokenized staking positions that accrue staking rewards while remaining fully liquid and transferable. Example: xBETH, xOKSOL.

## 2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

**Basic coding mistakes.** Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

**Business logic errors.** Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

**Integration risks.** Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

**Code maturity.** We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability

weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

## 2.3. Scope

The engagement involved a review of the following targets:

### xAsset Contracts

| | |
|---|---|
| **Type** | Solidity |
| **Platform** | EVM-compatible |
| **Target** | xAsset |
| **Repository** | https://github.com/okx/xAsset ↗ |
| **Version** | `e8a0b80f91490166ad3fc06a24e296d95c0515ad` |
| **Programs** | `contracts/*.sol` |

## 2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of three person-days. The assessment was conducted by two consultants over the course of two calendar days.

### Contact Information

The following project manager was associated with the engagement:

**Jacob Goreski**
Engagement Manager
jacob@zellic.io

The following consultants were engaged to conduct the assessment:

**Qingying Jie**
Engineer
qingying@zellic.io

**Varun Verma**
Engineer
varun@zellic.io

## 2.5. Project Timeline

The key dates of the engagement are detailed below.

| | |
|---|---|
| **December 8, 2025** | Start of primary review period |
| **December 9, 2025** | End of primary review period |

# 3. Detailed Findings

## 3.1. Accounts on the denylist can still transfer tokens

| Target | xbtc, xToken | | |
|---|---|---|---|
| **Category** | Business Logic | **Severity** | High |
| **Likelihood** | Medium | **Impact** | Medium |

### Description

Addresses on the denylist are not allowed to send or receive tokens. However, the function `_update`, which is used to update the account balance during token transfers, only checks whether the `from` and `to` addresses are on the denylist; it does not check `msg.sender`.

```
function _update(
    address from,
    address to,
    uint256 value
) internal override(ERC20Upgradeable, ERC20PausableUpgradeable) {
    if (denyList[from]) revert SenderInDenyList(from);
    if (denyList[to]) revert RecipientInDenyList(to);

    super._update(from, to, value);
}
```

### Impact

A malicious address on the denylist can use allowances previously granted by other accounts to transfer those accounts' tokens via the function `transferFrom`.

### Recommendations

Add a check to ensure that `msg.sender` is not on the denylist.

### Remediation

This issue has been acknowledged by OKX, and a fix was implemented in commit a5cc31a9 ↗.

### 3.2. The function `setReceiver` may set `authorizedReceiver` to an address that is on the denylist

| | | | |
|---|---|---|---|
| **Target** | xbtc, xToken | | |
| **Category** | Business Logic | **Severity** | Low |
| **Likelihood** | Low | **Impact** | Low |

#### Description

The function `setReceiver` does not check if the `newReceiver` is on the denylist.

```
function setReceiver(address newReceiver) public onlyRole(DENY_LISTER_ROLE) {
    if (newReceiver == address(0)) revert ZeroAddress();
    address previousReceiver = authorizedReceiver;
    authorizedReceiver = newReceiver;
    emit ReceiverSet(previousReceiver, newReceiver);
}
```

#### Impact

Because tokens can only be minted to the `authorizedReceiver` and addresses on the denylist cannot receive tokens, if the `authorizedReceiver` is mistakenly set to an address on the denylist, the `mint` function will be unusable until `authorizedReceiver` is reset to a valid address.

```
function mint(address receiver, uint256 amount)
public
onlyRole(MINTER_ROLE)
whenNotPaused
{
    if (amount == 0) revert ZeroAmount();
    // in this case, we double confirm the mint intention matched with the
    authorized receiver
    if (receiver != authorizedReceiver) revert NoAuthorizedReceiver();
    if (totalSupply() + amount > MAX_SUPPLY)
    revert ExceedsMaxSupply(totalSupply() + amount, MAX_SUPPLY);
    _mint(receiver, amount);
    emit Mint(receiver, amount);
}
```

### Recommendations

Consider adding a check to ensure that the `newReceiver` is not on the denylist.

### Remediation

This issue has been acknowledged by OKX.

OKX provided the following comment:

> `authorizedReceiver` should never in the denylist.

### 3.3. Exchange rate defaults to zero before oracle initialization

| | | | |
|---|---|---|---|
| **Target** | StakedTokenV1 | | |
| **Category** | Integration Risks | **Severity** | Low |
| **Likelihood** | Low | **Impact** | Low |

#### Description

The `exchangeRate()` function reads directly from an uninitialized storage slot and returns 0 until the oracle calls `updateExchangeRate()` for the first time.

```
function exchangeRate() public view returns (uint256 _exchangeRate) {
    bytes32 position = _EXCHANGE_RATE_POSITION;
    assembly {
        _exchangeRate := sload(position)
    }
}
```

Internally, the protocol only uses this value for equality checks and subtraction in `ExchangeRateUpdater`, so no division by zero occurs within the contract itself. However, the function is public, and external integrators could revert or produce unexpected results when the rate is uninitialized.

#### Impact

External protocols integrating with the staked token may experience reverts or incorrect calculations if they query the exchange rate before the oracle has set it, with no way to distinguish between the rate being zero and the rate being uninitialized.

#### Recommendations

Document that `exchangeRate()` returns 0 before the oracle's first update, and advise integrators to treat 0 as uninitialized rather than a valid rate.

#### Remediation

This issue has been acknowledged by OKX, and a fix was implemented in commit `dd5d9fd6` ↗.

OKX provided the following comment:

We will make sure initial exchange rate is set to 1e18 in one atomic deploy & initialize transaction.

## 3.4. The role-transfer operation may devolve into a revoke-role operation

| Target | xbtc, xToken | | |
|---|---|---|---|
| **Category** | Business Logic | **Severity** | Informational |
| **Likelihood** | Low | **Impact** | Informational |

### Description

Functions `transferMinter` and `transferDenyLister` do not check whether the target address already has the corresponding role, and the function `_grantRole` simply returns false if the target address has the specified role.

```
function transferMinter(address newMinter) public onlyRole(MINTER_ROLE) {
    if (newMinter == address(0)) revert ZeroAddress();
    if (newMinter == msg.sender) revert SameValue();

    address previousMinter = msg.sender;

    // Revoke minter role from current minter
    _revokeRole(MINTER_ROLE, previousMinter);

    // Grant minter role to new minter
    _grantRole(MINTER_ROLE, newMinter);

    emit MinterTransferred(previousMinter, newMinter);
}
```

### Impact

If the target address already has the corresponding role, then the caller is revoking their own role by calling these role-transfer functions, and the total number of privileged accounts will decrease.

### Recommendations

Consider checking the return value of the function `_grantRole` and reverting the transaction if the return value is false.

### Remediation

This issue has been acknowledged by OKX.

OKX provided the following comment:

> It's acceptable, normally there are only 1 minter and 1 denylister. Also, minter/denylister can revoke itself role.

# 4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

## 4.1. Incorrect decimal places in the comments

The contract xToken actually has 18 decimals, but the comments in functions `mint` and `burn` state that the parameter `amount` has eight decimal places, which could be misleading.

Consider updating the comments to 18 decimal places.

```
/**
* @dev Mints new tokens to the authorized receiver address
* @param amount The amount of tokens to mint (in 8 decimal places)
// [...]
function mint(address receiver, uint256 amount)
// [...]

/**
* @dev Burns tokens from the caller's balance
* @param amount The amount of tokens to burn (in 8 decimal places)
// [...]
function burn(uint256 amount)
// [...]
```

This issue has been acknowledged by OKX, and a fix was implemented in commit 72e0408a ↗.

## 4.2. Test suite

The test suite is well-structured and covers the core functionality thoroughly. Unit tests validate the expected behavior for minting, burning, pausing, denylist operations, and role management, with good coverage of both happy paths and revert conditions. The OKX team also includes integration tests that exercise the full oracle-to-token flow for exchange-rate updates as well as governance tests demonstrating timelock-controlled upgrades. The use of a test harness in the rate-limit tests to expose internal functions is beneficial for validating the allowance replenishment math.

Fuzz testing is present but fairly light. The existing fuzz tests cover basic mint/burn bounds and caller configuration, but more aggressive fuzzing around the rate-limit arithmetic and exchange-rate edge cases would strengthen confidence in those areas.

The main gap we identified is that the denylist tests only check that `from` and `to` addresses are

blocked and do not test whether a denylisted `msg.sender` can still call the function `transferFrom` using someone else's allowance. An invariant test asserting "denylisted addresses cannot interact with the token at all" would have caught this. There is also no coverage for edge cases like setting `authorizedReceiver` to a denylisted address or querying the function `exchangeRate` before the oracle has initialized it.

Overall, the test suite provides good baseline coverage and follows reasonable patterns but could benefit from more adversarial testing around access-control boundaries.

# 5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

## 5.1. Module: ExchangeRateUpdater.sol

### Function: `updateExchangeRate(uint256 _newExchangeRate)`

This function allows whitelisted callers to update the exchange rate of the `tokenContract`, subject to rate-limiting constraints.

#### Inputs

- `_newExchangeRate`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be greater than zero and different from the current exchange rate.
  - **Impact**: The new exchange rate to be set for the `tokenContract`.

#### Branches and code coverage

**Intended branches**

- The exchange rate of the `tokenContract` can be successfully updated.
  - ☑ Test coverage
- The caller's allowance is updated as expected.
  - ☑ Test coverage
- The `ExchangeRateUpdated` event is emitted with the caller address and the `_newExchangeRate`.
  - ☑ Test coverage

**Negative behavior**

- Reverts if the caller is not whitelisted.
  - ☑ Negative test
- Reverts if `_newExchangeRate` is zero.
  - ☑ Negative test

- Reverts if `_newExchangeRate` is the same as the current exchange rate.
  - ☑ Negative test
- Reverts if the difference between `_newExchangeRate` and the current exchange rate exceeds the caller's allowance.
  - ☑ Negative test

### Function call analysis

- `ExchangeRateUtil.safeGetExchangeRate(this.tokenContract)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** The return value is the current exchange rate of the `tokenContract`.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `ExchangeRateUtil.safeUpdateExchangeRate(_newExchangeRate, this.tokenContract)`
  - **What is controllable?** `_newExchangeRate`.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

## 5.2. Module: RateLimit.sol

### Function: `configureCaller(address caller, uint256 amount, uint256 interval)`

This function allows the contract owner to configure the rate-limit parameters for a specific address.

### Inputs

- `caller`
  - **Control**: Fully controlled by the owner.
  - **Constraints**: Must not be the zero address.
  - **Impact**: The address being configured.
- `amount`
  - **Control**: Fully controlled by the owner.
  - **Constraints**: Must be greater than zero.

    - **Impact**: The maximum allowance for the given `caller`.
- `interval`
    - **Control**: Fully controlled by the owner.
    - **Constraints**: Must be greater than zero.
    - **Impact**: The time required for the allowances of the `caller` to be refilled.

### Branches and code coverage

#### Intended branches

- The `callers[caller]` is set to true.
    - ☑ Test coverage
- The `maxAllowances` and `allowances` for the `caller` are set to `amount`.
    - ☑ Test coverage
- The `allowancesLastSet` for the `caller` is updated to the current `block.timestamp`.
    - ☑ Test coverage
- The `intervals` for the `caller` is set to `interval`.
    - ☑ Test coverage
- The `CallerConfigured` event is emitted with correct parameters.
    - ☑ Test coverage

#### Negative behavior

- Reverts if the `caller` is the zero address.
    - ☑ Negative test
- Reverts if `amount` is zero.
    - ☑ Negative test
- Reverts if `interval` is zero.
    - ☑ Negative test
- Reverts if the `msg.sender` is not the owner.
    - ☑ Negative test

## Function: `removeCaller(address caller)`

This function allows the contract owner to remove the rate-limit configuration for a specified address.

### Inputs

- `caller`
  - **Control**: Fully controlled by the owner.
  - **Constraints**: N/A.
  - **Impact**: The address being removed.

### Branches and code coverage

#### Intended branches

- The `callers`, `intervals`, `allowancesLastSet`, `maxAllowances`, and `allowances` mappings for the `caller` are cleared.
  - ☑ Test coverage
- The `CallerRemoved` event is emitted with the `caller` address.
  - ☑ Test coverage
- Must not revert if the `caller` does not exist.
  - ☑ Test coverage

#### Negative behavior

- Reverts if the `msg.sender` is not the owner.
  - ☑ Negative test

## 5.3. Module: StakedTokenV1.sol

### Function: `updateExchangeRate(uint256 newExchangeRate)`

This function allows the exchange-rate oracle to update the exchange rate.

### Inputs

- `newExchangeRate`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be greater than zero.
  - **Impact**: The new exchange rate.

### Branches and code coverage

#### Intended branches

- The exchange rate can be updated successfully.

  ☑ Test coverage
- The event `ExchangeRateUpdated` is emitted with the caller address and the new exchange rate upon a successful update.

  ☑ Test coverage

**Negative behavior**

- Reverts if `newExchangeRate` is zero.

  ☑ Negative test
- Reverts if the caller is not the oracle.

  ☑ Negative test

### Function: `updateOracle(address newOracle)`

This function allows an account with the `DEFAULT_ADMIN_ROLE` to update the address of the exchange-rate oracle.

### Inputs

- `newOracle`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address and must be different from the current oracle address.
  - **Impact**: The new exchange-rate oracle address.

### Branches and code coverage

**Intended branches**

- The oracle can be updated successfully.

  ☑ Test coverage
- The event `OracleUpdated` is emitted with the new oracle address upon a successful update.

  ☑ Test coverage

**Negative behavior**

- Reverts if `newOracle` is the zero address.

  ☑ Negative test

- Reverts if `newOracle` is the same as the current oracle.
  - ☑ Negative test
- Reverts if the caller does not have `DEFAULT_ADMIN_ROLE`.
  - ☑ Negative test

## 5.4. Module: xToken.sol

### Function: `addToDenyList(address account)`

This function allows an account with the `DENY_LISTER_ROLE` to add a single address to the denylist.

### Inputs

- `account`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address.
  - **Impact**: The address to be added to the denylist.

### Branches and code coverage

#### Intended branches

- The `account` is added to the denylist.
  - ☑ Test coverage
- The event `AddedToDenyList` is emitted with the account address.
  - ☑ Test coverage

#### Negative behavior

- Reverts if `account` is the zero address.
  - ☑ Negative test
- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

### Function: `batchAddToDenyList(address[] accounts)`

This function allows an account with `DENY_LISTER_ROLE` to add multiple addresses to the denylist.

### Inputs

- `accounts`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The array length must be greater than zero, and none of the addresses can be the zero address.
  - **Impact**: The addresses to be added to the denylist.

### Branches and code coverage

#### Intended branches

- Addresses in the `accounts` array are added to the denylist.
  - ☑ Test coverage
- The event `AddedToDenyList` is emitted for each address added.
  - ☐ Test coverage
- Addresses that are already in the denylist are skipped without error.
  - ☑ Test coverage

#### Negative behavior

- Reverts if the `accounts` array is empty.
  - ☑ Negative test
- Reverts if any address in the `accounts` array is the zero address.
  - ☑ Negative test
- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

## Function: `batchRemoveFromDenyList(address[] accounts)`

This function allows an account with `DENY_LISTER_ROLE` to remove multiple addresses from the denylist.

### Inputs

- `accounts`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The array length must be greater than zero.
  - **Impact**: The addresses to be removed from the denylist.

### Branches and code coverage

**Intended branches**

- Addresses in the `accounts` array are removed from the denylist.
  - ☑ Test coverage
- The event `RemovedFromDenyList` is emitted for each address removed.
  - ☐ Test coverage
- Addresses that are not in the denylist are skipped without error.
  - ☑ Test coverage

**Negative behavior**

- Reverts if the `accounts` array is empty.
  - ☑ Negative test
- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

## Function: `burn(uint256 amount)`

This function allows an account with the `MINTER_ROLE` to burn tokens from its own balance when the contract is not paused.

### Inputs

- `amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be greater than zero and not exceed the caller's balance.
  - **Impact**: The amount of tokens to burn.

### Branches and code coverage

**Intended branches**

- The expected amount of tokens is burned from the caller's balance.
  - ☑ Test coverage
- The event `Burn` is emitted with the caller's address and burned amount.
  - ☑ Test coverage

**Negative behavior**

- Reverts if `amount` is zero.
  - ☑ Negative test
- Reverts if the caller's balance is insufficient to cover the burn amount.
  - ☑ Negative test
- Reverts if the contract is paused.
  - ☑ Negative test
- Reverts if the caller does not have `MINTER_ROLE`.
  - ☑ Negative test
- Reverts if the caller is in the denylist.
  - ☐ Negative test

### Function: `mint(address receiver, uint256 amount)`

This function allows an account with the `MINTER_ROLE` to mint tokens to the authorized receiver address when the contract is not paused.

### Inputs

- `receiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be the `authorizedReceiver`.
  - **Impact**: The address that will receive the minted tokens.
- `amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be greater than zero and not cause the total supply to exceed `MAX_SUPPLY`.
  - **Impact**: The amount of tokens to mint.

### Branches and code coverage

#### Intended branches

- The expected amount of tokens is minted to the `authorizedReceiver`.
  - ☑ Test coverage
- The event `Mint` is emitted with the receiver address and minted amount.
  - ☑ Test coverage

**Negative behavior**

- Reverts if `amount` is zero.
  - ☑ Negative test
- Reverts if `receiver` is not the `authorizedReceiver`.
  - ☑ Negative test
- Reverts if the `authorizedReceiver` is in the denylist.
  - ☐ Negative test
- Reverts if the total supply after minting would exceed `MAX_SUPPLY`.
  - ☑ Negative test
- Reverts if the contract is paused.
  - ☑ Negative test
- Reverts if the caller does not have `MINTER_ROLE`.
  - ☑ Negative test

### Function: `removeFromDenyList(address account)`

This function allows an account with the `DENY_LISTER_ROLE` to remove a single address from the denylist.

### Inputs

- `account`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.
  - **Impact**: The address to be removed from the denylist.

### Branches and code coverage

**Intended branches**

- The `account` is removed from the denylist.
  - ☑ Test coverage
- The event `RemovedFromDenyList` is emitted with the account address.
  - ☑ Test coverage

**Negative behavior**

- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

### Function: `setReceiver(address newReceiver)`

This function allows an account with the `DENY_LISTER_ROLE` to set a new authorized receiver for minting operations.

#### Inputs

- `newReceiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address.
  - **Impact**: The address of the new authorized receiver.

#### Branches and code coverage

**Intended branches**

- The `authorizedReceiver` is updated successfully when a valid `newReceiver` is provided.
  - ☑ Test coverage
- The event `ReceiverSet` is emitted with the previous and new receiver addresses.
  - ☑ Test coverage

**Negative behavior**

- Reverts if `newReceiver` is the zero address.
  - ☑ Negative test
- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

### Function: `transferDenyLister(address newDenyLister)`

This function allows an account with the `DENY_LISTER_ROLE` to transfer the denylister role to a new address.

#### Inputs

- `newDenyLister`

- **Control**: Fully controlled by the caller.
- **Constraints**: Must not be the zero address and must not be the same as the caller.
- **Impact**: The address that will receive the denylister role.

### Branches and code coverage

#### Intended branches

- The caller's denylister role is revoked.
  - ☑ Test coverage
- The `newDenyLister` is granted the denylister role.
  - ☑ Test coverage
- The event `DenyListerTransferred` is emitted with the caller and new denylister addresses.
  - ☑ Test coverage

#### Negative behavior

- Reverts if `newDenyLister` is the zero address.
  - ☑ Negative test
- Reverts if `newDenyLister` is the same as the caller.
  - ☑ Negative test
- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

## Function: `transferMinter(address newMinter)`

This function allows an account with the `MINTER_ROLE` to transfer the minter role to a new address.

### Inputs

- `newMinter`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address and must not be the same as the caller.
  - **Impact**: The address that will receive the minter role.

### Branches and code coverage

**Intended branches**

- The caller's minter role is revoked.
  - ☑ Test coverage
- The `newMinter` is granted the minter role.
  - ☑ Test coverage
- The event `MinterTransferred` is emitted with the caller and new minter addresses.
  - ☑ Test coverage

**Negative behavior**

- Reverts if `newMinter` is the zero address.
  - ☑ Negative test
- Reverts if `newMinter` is the same as the caller.
  - ☑ Negative test
- Reverts if the caller does not have `MINTER_ROLE`.
  - ☑ Negative test

## Function: `_update(address from, address to, uint256 value)`

This internal function overrides the `_update` function of ERC20Upgradeable and ERC20PausableUpgradeable contracts to include checks against the denylist for both sender and recipient addresses before performing token transfers.

### Inputs

- `from`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be in the denylist.
  - **Impact**: The address that tokens are transferred from.
- `to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be in the denylist.
  - **Impact**: The address that tokens are transferred to.
- `value`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not exceed the token balance of the `from` address.

- **Impact**: The amount of tokens being transferred.

### Branches and code coverage

#### Intended branches

- The function successfully updates balances.
  - ☑ Test coverage

#### Negative behavior

- Reverts if the `from` address is in the denylist.
  - ☑ Negative test
- Reverts if the `to` address is in the denylist.
  - ☑ Negative test
- Reverts if the contract is paused.
  - ☐ Negative test

## 5.5. Module: xbtc.sol

### Function: `addToDenyList(address account)`

This function allows an account with the `DENY_LISTER_ROLE` to add a single address to the denylist.

### Inputs

- `account`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address.
  - **Impact**: The address to be added to the denylist.

### Branches and code coverage

#### Intended branches

- The `account` is added to the denylist.
  - ☑ Test coverage
- The event `AddedToDenyList` is emitted with the account address.
  - ☑ Test coverage

**Negative behavior**

- Reverts if `account` is the zero address.
  - ☑ Negative test
- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

### Function: `batchAddToDenyList(address[] accounts)`

This function allows an account with `DENY_LISTER_ROLE` to add multiple addresses to the denylist.

### Inputs

- `accounts`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The array length must be greater than zero, and none of the addresses can be the zero address.
  - **Impact**: The addresses to be added to the denylist.

### Branches and code coverage

**Intended branches**

- Addresses in the `accounts` array are added to the denylist.
  - ☑ Test coverage
- The event `AddedToDenyList` is emitted for each address added.
  - ☑ Test coverage
- Addresses that are already in the denylist are skipped without error.
  - ☑ Test coverage

**Negative behavior**

- Reverts if the `accounts` array is empty.
  - ☑ Negative test
- Reverts if any address in the `accounts` array is the zero address.
  - ☑ Negative test
- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

### Function: `batchRemoveFromDenyList(address[] accounts)`

This function allows an account with `DENY_LISTER_ROLE` to remove multiple addresses from the denylist.

#### Inputs

- `accounts`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: The array length must be greater than zero.
  - **Impact**: The addresses to be removed from the denylist.

#### Branches and code coverage

**Intended branches**

- Addresses in the `accounts` array are removed from the denylist.
  - ☑ Test coverage
- The event `RemovedFromDenyList` is emitted for each address removed.
  - ☑ Test coverage
- Addresses that are not in the denylist are skipped without error.
  - ☑ Test coverage

**Negative behavior**

- Reverts if the `accounts` array is empty.
  - ☑ Negative test
- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

### Function: `burn(uint256 amount)`

This function allows an account with the `MINTER_ROLE` to burn tokens from its own balance when the contract is not paused.

#### Inputs

- `amount`
  - **Control**: Fully controlled by the caller.

- **Constraints**: Must be greater than zero and not exceed the caller's balance.
- **Impact**: The amount of tokens to burn.

### Branches and code coverage

**Intended branches**

- The expected amount of tokens is burned from the caller's balance.
  - ☑ Test coverage
- The event `Burn` is emitted with the caller's address and burned amount.
  - ☑ Test coverage

**Negative behavior**

- Reverts if `amount` is zero.
  - ☑ Negative test
- Reverts if the caller's balance is insufficient to cover the burn amount.
  - ☑ Negative test
- Reverts if the contract is paused.
  - ☑ Negative test
- Reverts if the caller does not have `MINTER_ROLE`.
  - ☑ Negative test
- Reverts if the caller is in the denylist.
  - ☐ Negative test

## Function: `mint(address receiver, uint256 amount)`

This function allows an account with the `MINTER_ROLE` to mint tokens to the authorized receiver address when the contract is not paused.

### Inputs

- `receiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be the `authorizedReceiver`.
  - **Impact**: The address that will receive the minted tokens.
- `amount`
  - **Control**: Fully controlled by the caller.

- **Constraints**: Must be greater than zero and not cause the total supply to exceed `MAX_SUPPLY`.
- **Impact**: The amount of tokens to mint.

### Branches and code coverage

#### Intended branches

- The expected amount of tokens is minted to the `authorizedReceiver`.
  - ☑ Test coverage
- The event `Mint` is emitted with the receiver address and minted amount.
  - ☑ Test coverage

#### Negative behavior

- Reverts if `amount` is zero.
  - ☑ Negative test
- Reverts if `receiver` is not the `authorizedReceiver`.
  - ☑ Negative test
- Reverts if the `authorizedReceiver` is in the denylist.
  - ☑ Negative test
- Reverts if the total supply after minting would exceed `MAX_SUPPLY`.
  - ☑ Negative test
- Reverts if the contract is paused.
  - ☑ Negative test
- Reverts if the caller does not have `MINTER_ROLE`.
  - ☑ Negative test

## Function: `removeFromDenyList(address account)`

This function allows an account with the `DENY_LISTER_ROLE` to remove a single address from the denylist.

### Inputs

- `account`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: N/A.

- **Impact**: The address to be removed from the denylist.

### Branches and code coverage

#### Intended branches

- The `account` is removed from the denylist.
  - ☑ Test coverage
- The event `RemovedFromDenyList` is emitted with the account address.
  - ☑ Test coverage

#### Negative behavior

- Reverts if the caller does not have `DENY_LISTER_ROLE`.
  - ☑ Negative test

## Function: `setReceiver(address newReceiver)`

This function allows an account with the `DENY_LISTER_ROLE` to set a new authorized receiver for minting operations.

### Inputs

- `newReceiver`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address.
  - **Impact**: The address of the new authorized receiver.

### Branches and code coverage

#### Intended branches

- The `authorizedReceiver` is updated successfully when a valid `newReceiver` is provided.
  - ☑ Test coverage
- The event `ReceiverSet` is emitted with the previous and new receiver addresses.
  - ☑ Test coverage

#### Negative behavior

- Reverts if `newReceiver` is the zero address.

☑ Negative test

- Reverts if the caller does not have `DENY_LISTER_ROLE`.

☑ Negative test

### Function: `transferDenyLister(address newDenyLister)`

This function allows an account with the `DENY_LISTER_ROLE` to transfer the denylister role to a new address.

### Inputs

- `newDenyLister`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address and must not be the same as the caller.
  - **Impact**: The address that will receive the denylister role.

### Branches and code coverage

#### Intended branches

- The caller's denylister role is revoked.

☑ Test coverage

- The `newDenyLister` is granted the denylister role.

☑ Test coverage

- The event `DenyListerTransferred` is emitted with the caller and new denylister addresses.

☑ Test coverage

#### Negative behavior

- Reverts if `newDenyLister` is the zero address.

☑ Negative test

- Reverts if `newDenyLister` is the same as the caller.

☑ Negative test

- Reverts if the caller does not have `DENY_LISTER_ROLE`.

☑ Negative test

### Function: `transferMinter(address newMinter)`

This function allows an account with the `MINTER_ROLE` to transfer the minter role to a new address.

### Inputs

- `newMinter`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be the zero address and must not be the same as the caller.
  - **Impact**: The address that will receive the minter role.

### Branches and code coverage

**Intended branches**

- The caller's minter role is revoked.
  - ☑ Test coverage
- The `newMinter` is granted the minter role.
  - ☑ Test coverage
- The event `MinterTransferred` is emitted with the caller and new minter addresses.
  - ☑ Test coverage

**Negative behavior**

- Reverts if `newMinter` is the zero address.
  - ☑ Negative test
- Reverts if `newMinter` is the same as the caller.
  - ☑ Negative test
- Reverts if the caller does not have `MINTER_ROLE`.
  - ☑ Negative test

### Function: `_update(address from, address to, uint256 value)`

This internal function overrides the `_update` function of the ERC20Upgradeable and ERC20PausableUpgradeable contracts to include checks against the denylist for both sender and recipient addresses before performing token transfers.

### Inputs

- `from`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be in the denylist.
  - **Impact**: The address that tokens are transferred from.
- `to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not be in the denylist.
  - **Impact**: The address that tokens are transferred to.
- `value`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must not exceed the token balance of the `from` address.
  - **Impact**: The amount of tokens being transferred.

### Branches and code coverage

#### Intended branches

- The function successfully updates balances.
  - ☑ Test coverage

#### Negative behavior

- Reverts if the `from` address is in the denylist.
  - ☑ Negative test
- Reverts if the `to` address is in the denylist.
  - ☑ Negative test
- Reverts if the contract is paused.
  - ☑ Negative test

# 6. Assessment Results

During our assessment on the scoped xAsset contracts, we discovered four findings. No critical issues were found. One finding was of medium impact, two were of low impact, and the remaining finding was informational in nature.

## 6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.